for provenance such as dates and signatures. Similarly, the smart contract in a blockchain context implements the contract rules for solving a decentralized problem. It functions as a rules engine as well as a gatekeeper, so understandably, the smart contract design requires careful consideration. Following is an explanation of a smart contract modified to include the code aspects.

> **DEFINITION** A *smart contract* is executable code on the blockchain intended to digitally facilitate, verify, validate, and enforce the rules and regulations of an application. Smart contracts allow the performance of credible transactions without third parties. These transactions are trackable and irreversible.

In this chapter, you'll learn a set of design principles that will guide you through the design and development of smart contracts and blockchain programming. You'll apply these design principles to design a smart contract for a simple use case (a decentralized counter) and a different larger use case (for a decentralized airline consortium). To implement the design in the form of code, you need the following:

- A blockchain platform
- A language to code the smart contract
- A suitable environment to develop, compile, deploy, and test it

You'll use the Ethereum (https://ethereum.org) blockchain as the platform and a special language called Solidity (https://solidity.readthedocs.io/en/v0.6.2) to code the smart contracts. Then you'll deploy the code in an integrated development environment (IDE) called Remix (https://remix-ide.readthedocs.io/en/latest) and test its operation. This trio